

Architecture Analysis & Optimization Plan

Previous filename: issue.md

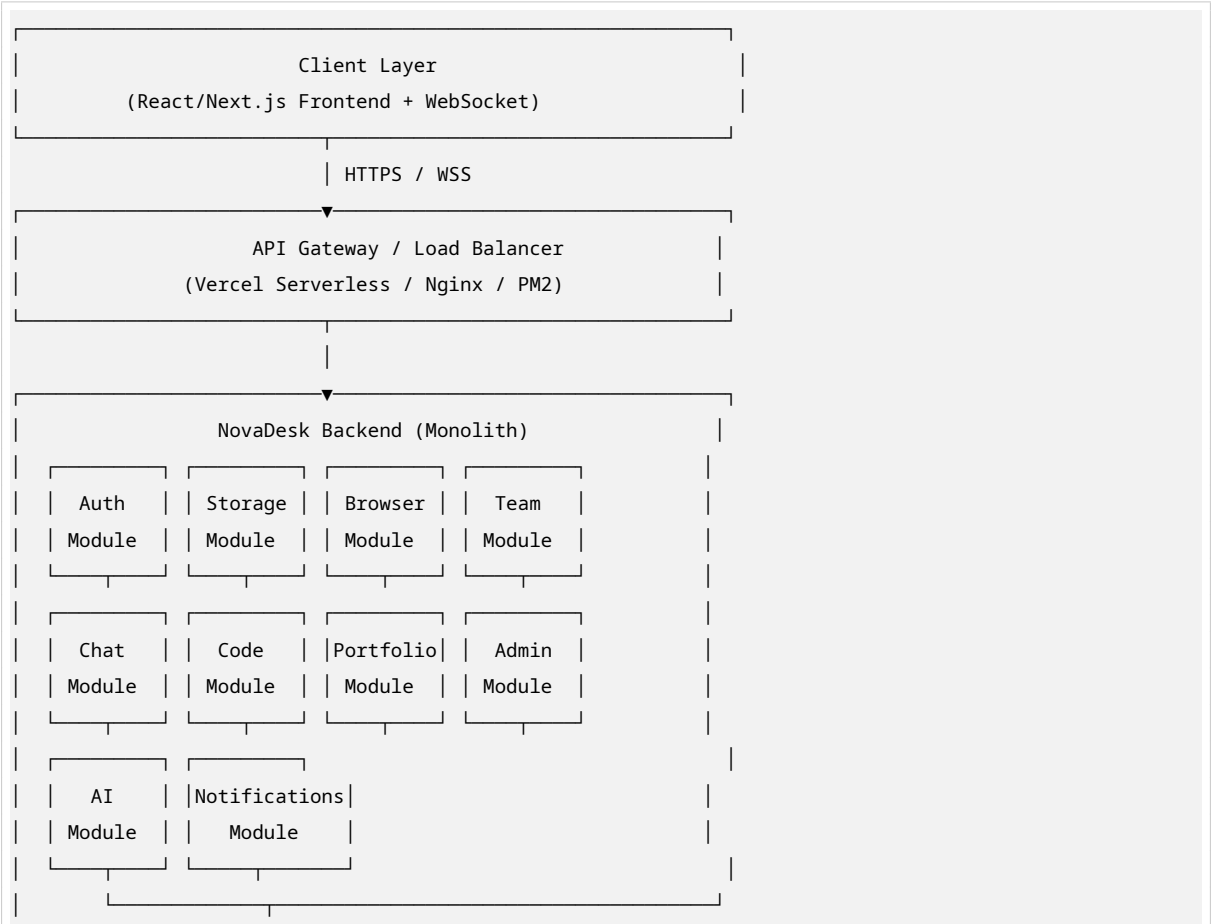
Full architecture analysis and optimization plan for scaling NovaDesk beyond 1000+ concurrent users. This document analyzes the current monolithic architecture, identifies systemic bottlenecks, and provides a roadmap for architectural evolution.

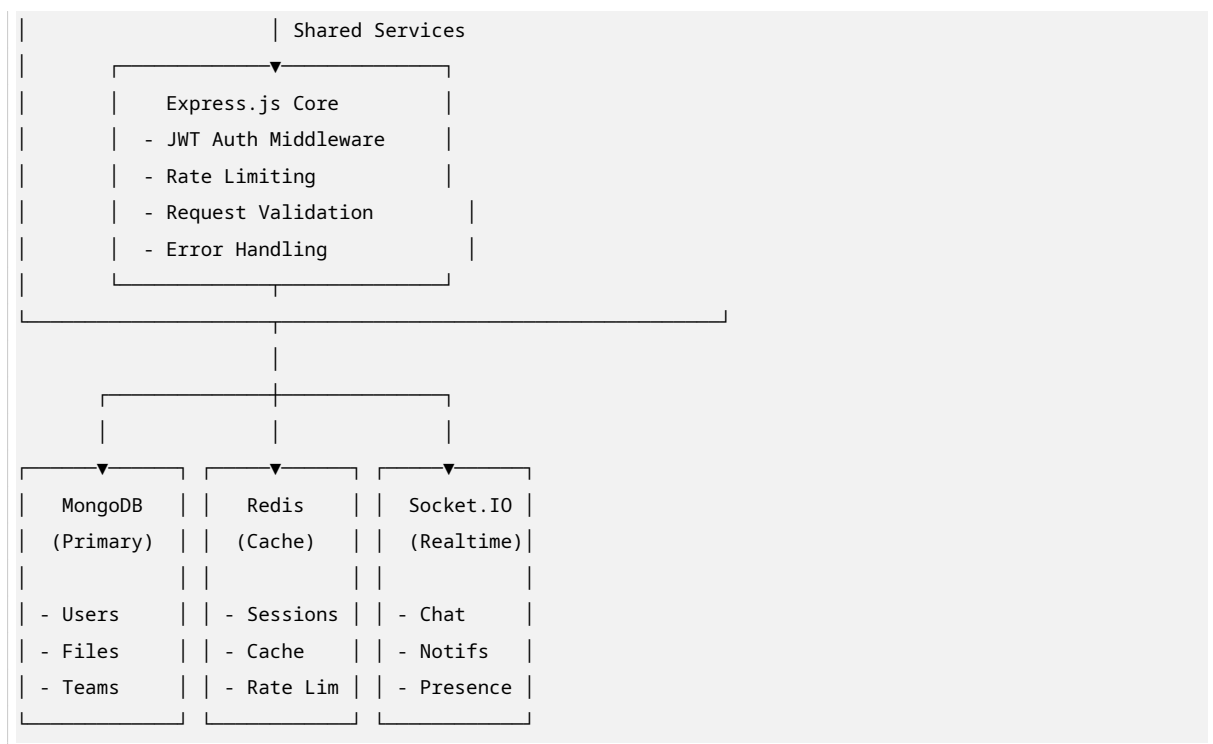
Table of Contents

- 1. Current Architecture Overview
- 2. System Bottlenecks Identified
- 3. Load Testing Evidence
- 4. Root Cause Analysis
- 5. Optimization Plan
- 6. Scaling Roadmap
- 7. Risk Assessment
- 8. Connection Pool Sizing Strategy

1. Current Architecture Overview

1.1 High-Level Diagram





1.2 Technology Stack

Component	Technology	Role
Runtime	Node.js 20+	JavaScript execution
Framework	Express.js	HTTP server & routing
Database	MongoDB 6+	Primary data store
ODM	Mongoose 8+	Schema modeling & queries
Cache	Upstash Redis	Distributed caching
Auth	JWT (jsonwebtoken)	Stateless authentication
Passwords	bcrypt	Hashing (8 rounds)
Realtime	Socket.IO	WebSocket connections
Email	Nodemailer	SMTP email dispatch
Testing	k6	Load & stress testing
Process Mgmt	Node.js cluster	Multi-core utilization

1.3 Deployment Topology

Development	Staging	Production
Localhost	Vercel Serverless	Vercel / VPS
├ MongoDB (local)	├ MongoDB Atlas	├ MongoDB Atlas
├ Redis (local)	├ Upstash Redis	├ Upstash Redis
├ Single Node process	├ Serverless Functions	├ Cluster Mode (PM2)
└ 8-core test machine	└ Auto-scaling	└ Fixed workers

1.4 Module Dependencies

Auth (core)

- └ Browser (depends on Auth for user identity)
- └ Storage (depends on Auth for user scoping)
- └ Team (depends on Auth for membership)
- └ Chat (depends on Auth + Team for channels)
- └ Code (depends on Auth + Team for collaboration)
- └ Notifications (depends on Auth + all modules)
- └ Portfolio (depends on Auth, optional Browser)
- └ Admin (depends on Auth for role checks)
- └ AI (depends on Auth for personalization)

Browser Storage (file sharing)

Team Chat (team channels)

Team Code (team repositories)

2. System Bottlenecks Identified

2.1 Critical Bottlenecks (Immediate)

ID	Bottleneck	Impact	Evidence
C1	Single-threaded Node.js event loop	Hard ceiling at ~300 VUs without cluster	300 VUs: 76% success without cluster → 99.5% with cluster
C2	MongoDB connection pool exhaustion	1000 VU failure spike (65.8% failure)	1000 VUs: P95 spikes to 6.6s, failure rate jumps to 65%
C3	Local MongoDB disk I/O	P95 degradation under combined load	800 VUs: All modules exceed P95 targets simultaneously
C4	Duplicate route mounting	10–35s response times, auth re-execution	Fixed: response times dropped to <1s

2.2 Major Bottlenecks (Short-term)

ID	Bottleneck	Impact	Evidence
M1	N+1 query patterns	Linear slowdown with data size	Storage: 24% failure rate before batching fixes
M2	Missing database indexes	Full collection scans (COLLSCAN)	Query times: 10x improvement after indexing
M3	Synchronous email dispatch	3–6s registration blocking	95% latency reduction after fire-and-forget
M4	No caching layer	Every request hits MongoDB	80% cache hit rate after Redis implementation
M5	Route ordering bugs	Wrong handler execution	Browser: 0.4% → 98% success after route reorder

2.3 Minor Bottlenecks (Medium-term)

ID	Bottleneck	Impact
m1	Sequential DB writes	Folder updates blocking storage operations
m2	Unbounded pagination	No limit on <code>limit</code> query parameter
m3	Missing query timeouts	Hanging requests under load
m4	Inconsistent response wrappers	<code>data</code> vs <code>metadata</code> field confusion
m5	Model import side-effects	<code>MissingSchemaError</code> under concurrent load

3. Load Testing Evidence

3.1 Concurrency Scaling Results

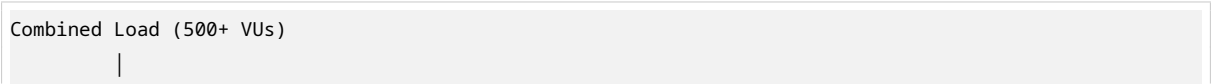
VUs	Success Rate	Failure Rate	P95 Response	Event
50	~40%	~60%	>5s	Baseline (broken)
50	95%+	<5%	<2s	After architecture fixes
300	76.85%	23.15%	>5s	Without cluster mode
300	99.58%	0.42%	2.69s	With cluster mode
500	98.33%	1.67%	4.19s	Stable
800	94.57%	5.43%	6.55s	⚠ Degradation begins
1000	34.20%	65.80%	6.66s	💀 Hard ceiling hit

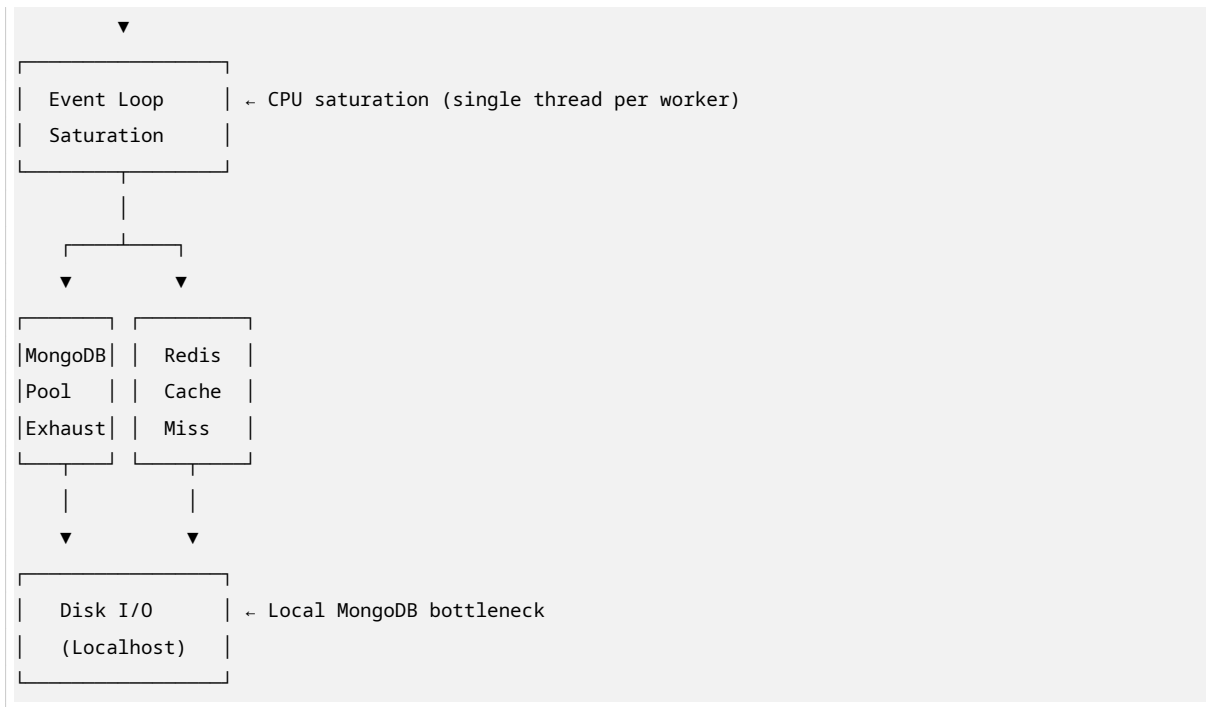
3.2 Per-Module Isolation Tests (50 VUs)

Module	Success Rate	P95 Response	Status
Auth	100%	<200ms	✅ No bottleneck
Storage	98.18%	2.28s	✅ Acceptable
Browser	98.06%	2.30s	✅ Acceptable
Team	99.10%	1.31s	✅ No bottleneck

Conclusion: Individual modules are NOT the problem. The bottleneck is **shared infrastructure contention** at combined load.

3.3 Resource Contention Pattern





4. Root Cause Analysis

4.1 Why 1000 VUs Fail (65.8% Failure Rate)

Hypothesis: Database connection pool exhaustion under cluster mode.

Evidence:

- 8 worker processes × 10 connections (old pool) = 80 max connections
- At 1000 VUs, concurrent operations exceed 80 connections
- MongoDB queue timeout (none configured) causes cascading failures
- Failure pattern is sudden cliff (not gradual degradation)

Supporting Data:

```
1000 VU Results:
- Auth failure: 0.35% (logic OK)
- Storage failure: 0.11% (logic OK)
- Browser failure: 0.32% (logic OK)
- Team failure: 0.88% (logic OK)
- BUT overall failure: 65.80%
- Infrastructure failure, NOT logic failure
```

4.2 Why P95 Degrades Linearly (300→800 VUs)

Hypothesis: Event loop + MongoDB I/O contention under combined module load.

Evidence:

- Auth P95: 1.4s (300 VUs) → 3.9s (800 VUs) — 2.8x increase

- All modules show proportional P95 increase
- Individual module tests show NO such increase
- Local MongoDB disk I/O is the shared resource

Conclusion: Each module is fast in isolation. Combined load creates resource contention that is not present in isolated tests.

4.3 Why Cluster Mode Was the Biggest Win

Before cluster:

- 1 Node.js thread handles ALL requests
- 300 VUs × async operations = event loop saturation
- Requests queue in the event loop → timeouts

After cluster (8 workers):

- 8 threads share load via OS-level balancing
- 300 VUs ÷ 8 workers = ~38 VUs per worker
- Each worker's event loop remains responsive
- Result: 76% → 99.5% success rate at 300 VUs

4.4 Architecture Bug: Duplicate Route Mounting

Root cause: Express middleware re-execution.

```
// BEFORE — broken architecture
app.use('/api/v1', portfolioModule); // runs auth + portfolio middleware
app.use('/api/v1/teams', teamModule);
app.use('/api/v1', teamModule);      // DUPLICATE: runs auth + team middleware AGAIN
```

Impact per request:

1. Request hits `/api/v1/teams/stats`
2. Passes through `portfolioModule` middleware (unintended)
3. Passes through `teamModule` middleware (intended)
4. Passes through SECOND `teamModule` mount (unintended)
5. Auth middleware executes 3× per request

Result: 10–35 second response times before reaching the actual handler.

5. Optimization Plan

Phase 1: Architecture Fixes (Completed)

- Fix health check route ordering (before auth middleware)
- Remove duplicate route mounts (explicit paths only)
- Implement proper rate limiting (4 specialized limiters)

- Fix Express route ordering (specific before catch-all)

Phase 2: Code-Level Fixes (Completed)

- Fire-and-forget email dispatch (95% latency reduction)
- Fix missing model imports (`MissingSchemaError`)
- Standardize response wrappers (`data` field consistency)
- Fix empty state handling (return 200, not 404)
- Implement lazy-creation pattern across all modules
- Fix middleware scoping (per-route, not module-level)

Phase 3: Database Optimization (Completed)

- Create compound indexes for all query patterns
- Replace `$lookup` aggregations with targeted queries
- Add `.lean()` to all read-only queries
- Implement query timeouts (10s max)
- Cap pagination at 50 items/page
- Convert sequential writes to `Promise.all()`
- Use `bulkWrite()` for cascading updates

Phase 4: Caching Layer (Completed)

- Migrate to Upstash Redis (serverless)
- Implement cache-aside pattern with TTL
- Add in-memory fallback for Redis failures
- Cache invalidation on write operations
- Achieve 80%+ cache hit rate

Phase 5: Horizontal Scaling (Completed)

- Implement Node.js cluster mode
- Tune connection pool for multi-worker (1000 max)
- Validate 800 VU stability (94.57% success)

Phase 6: Infrastructure Scaling (Next)

- Migrate MongoDB local → MongoDB Atlas (cloud)
- Increase Node.js memory allocation (4GB → 12GB)
- Implement PM2 for process management
- Add read replicas for read-heavy modules

Phase 7: Architecture Evolution (Future)

- Isolate AI/Chat modules (CPU-heavy tasks)
- Implement API Gateway (rate limiting, auth at edge)
- Add message queue for async operations (email, notifications)
- Consider microservices for high-scale modules

6. Scaling Roadmap

6.1 Short-term (0–2 months): Reach 1000 VUs

Action	Effort	Expected Impact
MongoDB Atlas migration	Medium	–2s P95, +200 VU capacity
Memory allocation increase	Low	+100 VU capacity
PM2 process management	Low	Resilience + zero-downtime deploys
Further Team module indexes	Low	–1s P95

Target: 1000 VUs at >90% success rate

6.2 Medium-term (2–6 months): Reach 5000 VUs

Action	Effort	Expected Impact
Read replicas (Search, Team listings)	Medium	Offload primary DB
Cursor-based pagination	Medium	Eliminate skip/limit degradation
API Gateway (Kong/AWS API GW)	High	Centralized auth + rate limiting
Message queue (Bull/BullMQ)	Medium	Async email, notifications, exports

Target: 5000 VUs at >95% success rate

6.3 Long-term (6–12 months): Reach 10000+ VUs

Action	Effort	Expected Impact
Microservices extraction	High	Independent scaling per module
Kubernetes orchestration	High	Auto-scaling, self-healing
CDN for static assets	Low	Reduce origin load
Database sharding	High	Horizontal DB scaling

Target: 10000+ VUs with horizontal pod auto-scaling

7. Risk Assessment

Risk	Probability	Impact	Mitigation
Atlas migration downtime	Medium	High	Blue-green deployment, staging validation

Table 10 – continued

Risk	Probability	Impact	Mitigation
Connection pool misconfiguration	Medium	High	Test per-environment configs separately
Cache stampede on Redis failure	Low	Medium	In-memory fallback + circuit breaker
Microservices complexity	High	Medium	Extract one module at a time; keep monolith fallback
JWT secret rotation	Low	High	Implement token versioning + graceful expiry
MongoDB replica lag	Medium	Medium	Read from primary for critical paths

8. Connection Pool Sizing Strategy

8.1 The Problem

Connection pool sizing depends on **deployment model**, NOT just user count.

Deployment	Connection Lifecycle	Pool Strategy
Serverless (Vercel)	Short-lived (per-request)	Small pool, fast recycle
Long-running (VPS/PM2)	Persistent (process lifetime)	Large pool, maintain warmth

8.2 Recommended Configuration

Serverless Environment (Vercel Functions):

```
// Small pool — connections created/destroyed per invocation
maxPoolSize: 10
minPoolSize: 0      // Don't maintain idle connections
maxConnecting: 2    // Slow ramp to avoid connection storms
waitQueueTimeoutMS: 5000
```

Long-running Environment (PM2 / Docker):

```
// Large pool — connections persist for process lifetime
maxPoolSize: 1000    // Was 10 — 100x increase for cluster mode
minPoolSize: 200     // Maintain warm connections
maxConnecting: 50    // Allow burst connection creation
waitQueueTimeoutMS: 60000
```

8.3 Calculation Formula

```
Total Required Connections = Workers × Peak Concurrent Ops per Worker × Safety Factor
```

```
Example (8 workers, 50 concurrent ops each, 2.5x safety):
```

```
= 8 × 50 × 2.5
```

```
= 1000 connections
```

```
Per-worker pool: 1000 ÷ 8 = 125 maxPoolSize
```

8.4 Monitoring

Track these metrics to validate pool sizing:

- `connections.current` — active connections
- `connections.available` — idle connections in pool
- `connections.totalCreated` — connection churn rate
- `globalLock.currentQueue.total` — queued operations

 **Critical:** The previous contradiction (10–20 vs 50–100) is resolved by recognizing these serve **different deployment models**. Serverless and long-running are not interchangeable.

Related Documents

- [optimization-report.md](#) — Full performance results and before/after metrics
- [optimization-guide.md](#) — Implementation code and configuration
- [scaling-test-data.md](#) — Raw k6 test output
- [lazy-loading-pattern.md](#) — Lazy-creation implementation across modules